☁️ MASTER AWS CLOUD MIGRATION MANUAL

Migrating Monthly Grocery from Vercel & Supabase to AWS

A zero-jargon, click-by-click engineering guide to migrate your entire production ecosystem — **PostgreSQL Database**, **Express Backend API**, **Next.js Web Admin**, and **S3 Media Storage** — into a fully dedicated AWS cloud environment.

📍 CURRENT ARCHITECTURE

- 🌐 **Web Admin:** Vercel (Next.js 16)
- ⚡ **Backend API:** Vercel Serverless / Local
- 🐘 **Database:** Supabase Cloud PostgreSQL
- 📁 **Storage:** Supabase Storage / Local

➔

DIRECT MOVE

🚀 TARGET AWS ARCHITECTURE

- 🖥️ **Backend Server:** AWS EC2 (Ubuntu 24.04 + PM2 + Nginx)
- 🐘 **Database:** Amazon RDS PostgreSQL (db.t4g.micro)
- 📦 **Storage:** Amazon S3 + CloudFront CDN
- 🌐 **Web Admin:** AWS Amplify or Same EC2 (Port 3000)

MIGRATION STEPS

🖨️ Print / Save Guide as PDF

1 AWS Account Setup & Security Firewall

Establish your AWS foundation and configure ports for HTTP, HTTPS, SSH, and Database

1.1 Select AWS Region & Create Security Group

For Indian grocery customers, always select the **Asia Pacific (Mumbai) `ap-south-1`** region for the lowest latency (~20ms).

Console Steps:

1. Log in to the **AWS Management Console** (console.aws.amazon.com).
2. In the top-right header, verify the Region is set to **Mumbai (ap-south-1)**.
3. Search for **EC2** in the top search bar > Click **Security Groups** (under Network & Security on the left menu).
4. Click **Create security group** > Name it: `monthlygrocery-ec2-sg`.
5. Add the following **Inbound rules**:

Type	Protocol	Port Range	Source	Purpose
SSH	TCP	22	`My IP` (or 0.0.0.0/0)	Server terminal access
HTTP	TCP	80	`0.0.0.0/0`	Standard web traffic & SSL verification
HTTPS	TCP	443	`0.0.0.0/0`	Encrypted API & Web Admin traffic

- [ ] ✓ Step 1 Complete: AWS Region (ap-south-1) confirmed & Security Group created

2 Migrate Supabase PostgreSQL ➔ AWS RDS

Export all tables, sequences, orders, products, and profiles to a managed Amazon RDS database

2.1 Create Amazon RDS PostgreSQL Instance

Console Steps:

1. Go to **RDS** in AWS Console > Click **Create database**.
2. Choose **Standard create** > Engine: **PostgreSQL** (Version: 16 or 15).
3. Template: **Free tier** (or Dev/Test).
4. DB instance identifier: `monthlygrocery-prod-db`.
5. Master username: `postgres` (or `mgadmin`).
6. Master password: *Set a strong 20-character password and save it securely!*
7. Instance configuration: **db.t4g.micro** (Burstable class).
8. Storage: **20 GB General Purpose SSD (gp3)** with auto-scaling enabled.
9. Connectivity:
   - VPC: **Default VPC**.
   - Public access: **Yes** (temporarily for data migration, then set to No).
   - VPC security group: Select the `monthlygrocery-ec2-sg`.
10. Click **Create database** (takes ~5–7 minutes to become "Available").

2.2 Export Data from Supabase

Run this command on your local machine to dump all tables (`products`, `shops`, `orders`, `profiles`, `delivery_slots`, `coupons`) from Supabase:

Command Prompt / Terminal (Local Machine)

```
# 1. Export schema and data from your Supabase PostgreSQL DB
pg_dump -h db.xlnebedclqcmgfbfqkbm.supabase.co -U postgres -p 5432 -d postgres --clean --if-exists --no-owner --no-privileges
```

2.3 Import Data into Amazon RDS

Copy the Endpoint URL from your RDS Dashboard (e.g. `monthlygrocery-prod-db.c3xxx.ap-south-1.rds.amazonaws.com`) and run:

Command Prompt / Terminal (Local Machine)

```
# 2. Restore all data into your new AWS RDS PostgreSQL database
psql -h monthlygrocery-prod-db.c3xxx.ap-south-1.rds.amazonaws.com -U postgres -d postgres -f monthlygrocery_backup.sql
```

✅ **Verification:** Log in with `psql` and run `\dt`. You will see all 8+ core tables transferred cleanly with zero data loss.

☐ ✓ Step 2 Complete: Database migrated from Supabase to AWS RDS PostgreSQL

3 Create Amazon S3 Bucket for Media & Images

Stores product photos, merchant banners, user avatars, and Excel catalog uploads

3.1 Create S3 Bucket

Console Steps:

1. Go to **Amazon S3** > Click **Create bucket**.
2. Bucket name: `monthlygrocery-prod-assets-2026` (Must be globally unique).
3. Region: **Asia Pacific (Mumbai) ap-south-1**.
4. Object Ownership: **ACLs enabled** (or Bucket owner preferred).
5. Block Public Access: Uncheck *"Block all public access"* so product images can be viewed publicly by customer apps.
6. Click **Create bucket**.

3.2 Generate IAM Access Keys for Backend

Console Steps:

1. Go to **IAM** > **Users** > Click **Create user**.
2. User name: `monthlygrocery-backend-uploader`.
3. Attach policies directly > Select **AmazonS3FullAccess**.
4. Click **Create user** > Go to the user > **Security credentials** tab.
5. Click **Create access key** > Select *"Application running outside AWS"*.
6. Copy and save your **Access Key ID** and **Secret Access Key**.

☐ ✓ Step 3 Complete: S3 Bucket created and IAM upload credentials generated

4 Launch AWS EC2 Virtual Server & Deploy Backend API

Set up Ubuntu 24.04 LTS, Node.js 20/22, PM2 daemon, and run the Express server 24/7

4.1 Launch EC2 Instance

Console Steps:

1. Go to **EC2** > Click **Launch instance**.
2. Name: `MonthlyGrocery-Production-Server`.
3. OS: **Ubuntu Server 24.04 LTS (HVM), SSD Volume Type**.
4. Instance type: **t4g.small** (or **t3.small / t3.medium**).
5. Key pair: Click *Create new key pair* > Name: `monthlygrocery-key` > Format: `.pem` (Save to your PC).
6. Network settings: Select existing Security Group: `monthlygrocery-ec2-sg`.
7. Storage: **30 GB gp3** root volume.
8. Click **Launch instance**.

4.2 SSH into Server & Install Node.js + PM2

Terminal / PowerShell

```
# 1. Connect to your EC2 instance
ssh -i "monthlygrocery-key.pem" ubuntu@YOUR_EC2_PUBLIC_IP

# 2. Update system packages
sudo apt update && sudo apt upgrade -y

# 3. Install Node.js 20 LTS and Git
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install -y nodejs git build-essential nginx

# 4. Install PM2 Process Manager globally (to keep API alive 24/7)
sudo npm install -g pm2 typescript
```

4.3 Clone Project & Configure Production .env

EC2 Terminal

```
# 1. Clone repository into /var/www
sudo mkdir -p /var/www && sudo chown -R ubuntu:ubuntu /var/www
cd /var/www
git clone https://github.com/your-org/MonthlyGrocery.git
cd MonthlyGrocery/express-backend

# 2. Install dependencies & build TypeScript
npm install
npm run build

# 3. Create production .env file
nano .env
```

Paste the following configuration into your server's `.env`:

/var/www/MonthlyGrocery/express-backend/.env

```
PORT=8001
NODE_ENV=production

# AWS RDS PostgreSQL Connection
DATABASE_URL=postgresql://postgres:YOUR_RDS_PASSWORD@monthlygrocery-prod-db.c3xxx.ap-south-1.rds.amazonaws.com:5432/postgres
```

```
# Security Secrets
JWT_SECRET=super-secure-production-jwt-random-key-2026-mg
DEV_OTP_BYPASS=false

# AWS S3 Storage
AWS_ACCESS_KEY_ID=YOUR_IAM_ACCESS_KEY
AWS_SECRET_ACCESS_KEY=YOUR_IAM_SECRET_KEY
AWS_REGION=ap-south-1
AWS_S3_BUCKET=monthlygrocery-prod-assets-2026

# Business Constraints
MIN_ORDER_LIMIT=3000
```

EC2 Terminal - Launch Backend with PM2

```
# Start the backend server
pm2 start dist/server.js --name "monthlygrocery-api"

# Save PM2 state to auto-restart on server reboot
pm2 startup
# (Copy & run the command outputted by pm2 startup, then run:)
pm2 save
```

- [ ] ✓ Step 4 Complete: Backend running 24/7 on EC2 with PM2

5 Configure Nginx Reverse Proxy & Free HTTPS SSL

Routes domain traffic (api.monthlygrocery.in) to port 8001 with Let's Encrypt SSL

5.1 Configure Nginx for API Subdomain

EC2 Terminal

```
sudo nano /etc/nginx/sites-available/monthlygrocery-api
```

Paste this Nginx server block:

/etc/nginx/sites-available/monthlygrocery-api

```
server {
    listen 80;
    server_name api.monthlygrocery.in;

    location / {
        proxy_pass http://127.0.0.1:8001;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection 'upgrade';
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_cache_bypass $http_upgrade;
        client_max_body_size 50M;
    }
}
```

EC2 Terminal - Enable Nginx Site & Issue Free SSL

```
# Enable config
sudo ln -s /etc/nginx/sites-available/monthlygrocery-api /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl restart nginx

# Install Certbot and generate SSL certificate
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx -d api.monthlygrocery.in
```

- [ ] ✓ Step 5 Complete: Nginx reverse proxy enabled with automated HTTPS SSL

6 Deploy Next.js 16 Web Admin Portal

Host the Super Admin dashboard on AWS (via AWS Amplify or on EC2)

Option A (Recommended) Deploy via AWS Amplify (Zero Server Maintenance)

Console Steps:

1. Go to **AWS Amplify** in AWS Console > Click **Create new app**.
2. Select **GitHub** > Connect your repository > Select branch `main`.
3. App root directory: set to `web-admin`.
4. Environment variables: Add `NEXT_PUBLIC_API_URL = https://api.monthlygrocery.in/api`.
5. Click **Save and deploy**.
6. In Domain management > Add custom domain: `admin.monthlygrocery.in` (Amplify automatically creates free SSL!).

Option B Deploy on Same EC2 Instance via PM2 (All-in-One)

EC2 Terminal

```
cd /var/www/MonthlyGrocery/web-admin
npm install
npm run build

# Run Next.js production server on port 3000 with PM2
pm2 start npm --name "web-admin" -- start -- -p 3000
pm2 save
```

- [ ] ✓ Step 6 Complete: Web Admin deployed and live on custom domain

7 Update React Native Mobile Apps (Customer & Merchant)

Switch API endpoints from local/Vercel URLs to your new production AWS server

7.1 Update API Base URL in React Native

In both `mobile-app` and `merchant-app`, update your config file (e.g. `src/config/api.ts`):

mobile-app/src/config/api.ts & merchant-app/src/config/api.ts

```
// Production AWS Live API Endpoint
export const API_BASE_URL = __DEV__
  ? 'http://10.0.2.2:8001/api'          // Android emulator local debug
  : 'https://api.monthlygrocery.in/api'; // AWS Production Server
```

- [ ] ✓ Step 7 Complete: Mobile apps configured with production AWS URL

8 DNS Cutover & Final Live Verification

Point your domain records to AWS and verify end-to-end grocery ordering

8.1 Update DNS Records (Cloudflare / GoDaddy / Namecheap)

Type	Name / Host	Value / Target
A Record	`api`	YOUR_EC2_ELASTIC_IP
CNAME Record	`admin`	YOUR_AMPLIFY_DOMAIN.amplifyapp.com

8.2 Live System Health Checklist

- 🩺 **Health Check:** Visit `https://api.monthlygrocery.in/health` or test an auth API call.
- 📱 **Customer App Test:** Open Customer app > Log in with OTP > Add items to Monthly Basket > Select Delivery Slot > Place COD Order.
- 🏪 **Merchant Partner App Test:** Open Merchant app > Verify incoming order notification > Change order status to "Out for Delivery".
- 💻 **Web Admin Portal Test:** Open `admin.monthlygrocery.in` > Check orders dashboard, inventory quantities, and revenue analytics.

- [ ] ✓ Step 8 Complete: DNS active, all health checks passed, platform is 100% live on AWS!